

VYSOKÉ UČENÍ TECHNICKÉ V BRNĚ

Fakulta informačních technologií

Technická správa

Aplikace pro získání statistik o síťovém provozu

Obsah

1	Úvod	3
2	Návod na použitie	3
3	Návrh aplikácie	3
3.1	Kontrola argumentov	3
3.2	Zachytávanie paketov	4
3.3	Analýza paketov	4
3.4	Ukladanie informácií o spojeniach	4
3.5	Rozdelenie programu do dvoch vlákien a výpis štatistík	4
4	Popis implementácie	4
4.1	arg_parse.c	4
4.2	isa-top.c	5
4.3	hashtable.c	5
4.3.1	Štruktúra	5
4.3.2	Funkcionalita	5
4.4	help.c	6
4.5	printer.c	6
5	Testovanie	6
5.1	Porovnanie výstupov s nástrojom iftop	6
5.2	Ping	7

1 Úvod

Nástroj isa-top slúži na monitorovanie sieťovej aktivity v reálnom čase a umožňuje sledovať prenosové rýchlosti aktuálnych sieťových pripojení.

Implementácia je realizovaná v jazyku C. Na zachytávanie a analýzu paketov je využitá knižnica libpcap a pre zobrazovanie a výpis štatistík knižnica ncurses.

Používateľ môže pomocou prepínačov pri spúšťaní špecifikovať sieťové rozhranie, na ktorom budú pakety zachytávané, takisto špecifikovať frekvenciu obnovovania štatistík a spôsob zoradenia výsledných štatistík.

Nástroj podporuje monitorovanie IPv4 aj IPv6 spojení, dokáže rozlišovať medzi TCP, UDP, ICMP a ICMPv6 protokolmi. Výsledky sú pravidelne vypisované vo formátovanej tabuľke, ktorá zobrazuje informácie ako IP adresy, porty, protokoly a prenosové rýchlosti pre maximálne 10 najrušnejších spojení na definovanom rozhraní.

Aplikácia bola vyvíjaná a testovaná na operačnom systéme Ubuntu 20.04.

2 Návod na použitie

Syntax spustenia programu je nasledujúca:

```
sudo ./isa-top -i INTERFACE [-t TIME] [-s b/p]
```

- `-i INTERFACE`

Povinný argument, ktorý predstavuje sieťové rozhranie, na ktorom sa budú packety zachytávať.

- `-t TIME`

Voliteľný argument, ktorý udáva ako často sa štatistiky budú aktualizovať, resp. na akom časovom intervale bude meraná rýchlosť prenosu. V prípade, že tento argument nie je pri spustení použitý, bude nastavený implicitný čas aktualizácie štatistík na 1 sekundu.

- `-s b/p`

Voliteľný argument, ktorý určuje v akom poradí budú jednotlivé spojenia zoradené vo výpise štatistík. Pri použití `-s b` budú spojenia zoradené zostupne podľa preneseného počtu bitov za sekundu alebo pri použití `-s p` budú spojenia zoradené podľa počtu paketov prenesených za sekundu, v rámci jednotlivých spojení. Implicitne, ak tento prepínač nie je využitý, program zoradí spojenia akoby bol použitý `-s b`.

V prípade nesprávneho spustenia aplikácie sa užívateľovi na chybovom výstupe zobrazí stručný návod na použitie, ktorý obsahuje aj výpis všetkých momentálne dostupných sieťových rozhraní.

Program je možné ukončiť stlačením **Ctrl+C**. Táto kombinácia odošle programu signál SIGINT, ktorý spustí upratanie a uvoľnenie zdrojov pred jeho ukončením.

3 Návrh aplikácie

Táto sekcia popisuje základné princípy a dôležité časti logickej štruktúry programu.

3.1 Kontrola argumentov

Prvým krokom je kontrola argumentov zadaných pri spustení. Argumenty sú validované pomocou knižnice getopt.h, aby bolo zaistené, že sú v súlade s pravidlami popísanými v sekcii 2.

3.2 Zachytávanie paketov

Pre zachytávanie a analýzu paketov je využitá knižnica `pcap.h`, poskytujúca štandardné funkcie pre interakciu s rozhraniami na nízkej úrovni. Kľúčovými časťami pri zachytávaní paketov sú 1) **vyhľadanie dostupných rozhraní**, 2) **získanie IP adresy** spolu s **maskou siete**, 3) **otvorenie session**, pričom aplikácia využíva tzv. `promiscuous` mód, čo znamená, že proces zachytávania paketov nebude limitovaný na pakety odoslané adresovanému zariadeniu, resp. budú sa zachytávať všetky pakety na sieti, 4) **Nastavená filtra** spolu s **kompiláciou**, 5) **Odchytávanie jednotlivých paketov** a ich **analýza** pomocou funkcie `pcap_loop`.

Časť kódu využívajúce knižnicu `pcap.h` a postup inšpirovaný príkladmi na zachytávanie paketov boli prevzaté z dokumentácie projektu `libpcap`, dostupnej na <https://www.tcpdump.org/pcap.html>. Tento projekt poskytuje dokumentáciu a príklady pod licenciou BSD, ktorá umožňuje používať, upravovať a šíriť kód s podmienkou uvedenia odkazu na pôvodný zdroj a licenciu.

3.3 Analýza paketov

Jednotlivé pakety sú analyzované v `callback` funkcií funkcii `pcap_loop`, kde je každý z týchto paketov rozdelený po hlavičkách a na každej úrovni sú získané informácie, ktoré sú pre funkčnosť programu potrebné. Takýmito informáciami sú najmä zistenie veľkosti paketu, ďalej názvy zdrojovej a cieľovej IP adresy spolu so zdrojovým a cieľovým portom.

3.4 Ukladanie informácií o spojeniach

Počas analýzy paketov je pre každý paket zistená adresa a port odosielateľa a prijímateľa, ktorou definujeme spojenie, resp. definujeme medzi ktorými účastníkmi prebieha komunikácia. Aby mohli byť informácie o jednotlivých spojeniach, na ktorých sú merané prenosové rýchlosti, efektívne a dynamicky ukladané, je využitá hashovacia tabuľka. Kľúče tejto tabuľky sú tvorené z IP adres a portov oboch účastníkov komunikácie, ukladanie položiek je nezávislé na smere komunikácie, teda nezávislé na tom, ktorý z účastníkov je odosielateľ a prijímateľ v rámci komunikácie, v hashovacej tabuľke sa stále bude jednať o rovnakú položku.

3.5 Rozdelenie programu do dvoch vlákien a výpis štatistík

Program neustále vykonáva dva hlavné procesy - analýzu prichádzajúcich paketov a výpis štatistík. Z tohto dôvodu je činnosť programu rozdelená do dvoch vlákien, z ktorých jedno vlákno slúži na nekonečné vykonávanie cyklu `pcap_loop`, resp. nekonečné zachytávanie a analýzu paketov, a druhé vlákno na opakovaný výpis štatistík, ktorého interval je určený prepínačom `-t` (alebo implicitne 1 sekunda). Pod pojmom výpis štatistík je v kontexte tohoto programu myslený výpis informácií o spojeniach, ako ich prijímacie a odosielačie prenosové rýchlosti a typ protokolu.

4 Popis implementácie

Program je implementovaný v jazyku C, každý `.c` zdrojový súbor má odpovedajúci `.h` hlavičkový súbor, v ktorom sú deklarované funkcie a/alebo dátové štruktúry daného `.c` súboru. Každá z následujúcich podsekcí obsahuje podstatné informácie o implementácii jednotlivých zdrojových súborov.

4.1 `arg_parse.c`

Tento súbor prevádza spracovanie argumentov zadáných pri spustení programu. Hlavnou a jedinou funkciou implementovanou v tomto súbore je `parse_arguments`. Pre spracovanie a kontrolu argumentov využíva funkciu `getopt_long` z knižnice `getopt.h`, to znamená, že všetky prepínače je pri spúšťaní programu

možné zadať aj v dlhšej verzii, pre prepínač `-i` to je `--interface`, pre `-t` `--time`, pre `-s` `--sort`. Funkcia `parse_arguments` vracia hodnotu 1, ak všetko prebehlo v poriadku, naopak, pri chybnom zadaní argumentov je navrátená hodnota 0 spolu so stručným popisom informácie na chybovom výstupe.

4.2 isa-top.c

Súbor `isa-top.c` implementuje jadro programu, nachádza sa v ňom `main` funkcia a implementuje logiku na zachytávanie a analýzu paketov, správu vlákien a uvoľňovania zdrojov.

Hlavná funkcia `main` má za úlohu inicializovať program a jeho dátové štruktúry, ďalej za použitia knižnice `pcap.h` nastaviť rozhranie a prostredie pre zachytávanie a analýzu paketov a takisto vytvorenie vlákien pre výpis štatistík a zachytávanie paketov.

Analýza paketov prebieha vo funkcii `packet_analyzer`, kde sú zachytené a následne do hashovacej tabuľky zapisované podstatné informácie o paketoch. Veľkosť celého paketu v bytoch je zistená z hlavičky linkovej vrstvy, pričom je od celkovej veľkosti odpočítaná veľkosť Ethernetovej hlavičky (14 bytov). Informácie o IP adresách a protokole sú zistené z IP hlavičky a informácie o portoch z transportnej vrstvy (TCP alebo UDP).

Funkcia `cleanup` slúži pre zachytenie signálu a bezpečné ukončenie programu. Po zachytení signálu ukončenia (Ctrl+C) uvoľní hashovaciu tabuľku, zavrie session handle a ukončí `ncurses` prostredie.

Funkcia `packet_capture_thread` implementuje logiku vlákna pre zachytávanie paketov, t.j. zaručuje nekonečnú analýzu paketov v `packet_analyzer` a `stats_print_thread` implementuje logiku vlákna pre výpis štatistík, to znamená, že v nekonečnom cykle sa opakuje uspanie vlákna na čas daný prepínačom `-t`, výpis štatistík a následne priamo po vypise je hashovacia tabuľka zrušená, aby pri ďalšom výpise obsahovala iba aktuálne spojenia a informácie.

4.3 hashtable.c

4.3.1 Štruktúra

Štruktúra položky v hashovacej tabuľke obsahuje nasledujúce hodnoty: zdrojovú IP `src_ip` alebo `src_ip6`, cieľovú IP `dst_ip` alebo `dst_ip6`, zdrojový port `src_port`, cieľový port `dst_port`, protokol `proto`, počet prijatých bitov `received_total_bits`, počet odoslaných bitov `transmit_total_bits`, počet prijatých paketov `received_total_packets`, počet odoslaných paketov `transmit_total_packets`, rýchlosť prijímania v bitoch `rx`, rýchlosť prijímania v paketoch `rx_packets`, rýchlosť odosielania v bitoch `tx`, rýchlosť odosielania v paketoch `tx_packets` a príznak či sa jedná o spojenie s IPv4 alebo IPv6 adresami `is_ip6`.

4.3.2 Funkcionalita

Súbor obsahuje implementáciu základných funkcií hashovacej tabuľky, konkrétne **inicializáciu** tabuľky - `htb_init`, **vkładanie** do tabuľky - `htb_insert`, **vyhľadanie** položky v tabuľke - `htb_search`, **aktualizáciu** hodnôt položky v tabuľke - `htb_update`, ďalej **zničenie**, resp. **uvoľnenie** položiek v tabuľke - `htb_destroy` a samozrejme samotnú **hashovaciu funkciu** pre priradenie kľúča do tabuľky - `hash_function`.

Hodnota navrátená z hashovacej funkcie `hash_function` je spojením zdrojových a cieľových IP adries a (spojené logickým operátorom XOR) portov modulo veľkosť tabuľky, ktorá je predom definovaná v kóde ako globálna premenná s hodnotou 1024. V prípade, že sa jedná o spojenie s IPv6 adresami, zoberie prvý bajt zdrojovej aj cieľovej IP adresy a vykoná medzi nimi bitový XOR, takú istú operáciu vykoná medzi zdrojovým a cieľovým portom a medzi výsledkami týchto dvoch operácií je vykonaný záverečný xor a z tohoto výsledku sa vykoná výpočet modulo veľkosť tabuľky. V prípade spojenia s IPv4 adresami je postup rovnaký, ale namiesto prvého bajtu sa berie v úvahu celá IPv4 adresa.

Funkcia pre vyhľadanie položky `htb_search` navracia nájdenu položku nezávisle na smere spojenia. To znamená, že ak tabuľka obsahuje záznam s rovnakými adresami a portami, avšak v obrátenom smere

komunikácie (t. j. zdrojová adresa a port sú zhodné s cieľovou adresou a portom a naopak), funkcia túto položku identifikuje ako zodpovedajúcu požadovanému spojeniu.

Funkcia pre vkladania položky `htb_insert` sa na začiatku pokúsi vyhľadať požadovanú položku (pomocou `htb_search`), v prípade, že je položka nájdená, je navrátená táto existujúca položka, inak je vytvorená nová položka a jej hodnoty sú nastavené na počiatočné.

Poslednou veľmi podstatnou funkciou hashovacej tabuľky, ktorej popis je nutný, je funkcia pre aktualizáciu hodnôt položiek `htb_update`. Jej podstatou je udržiavanie počtu prenesených bitov a paketov existujúcich spojení a výpočet prenosových rýchlostí. Funkcia najskôr položku vyhľadá, prípadne vloží do tabuľky, následne skontroluje smer komunikácie a na základe smeru (ktorý udáva či sa jedná o prijímanie alebo odosielanie) priradí do náležitých premenných počet prenesených bitov a paketov. Hneď po uložení počtu bitov a paketov sa vypočíta rýchlosť, ako počet bitov či paketov deleno čas obnovovania štatistík `packet_time`, ktorý je daný prepínom `-t` alebo implicitne 1 (sekunda).

4.4 help.c

V tomto súbore je implementovaná funkcia pre výpis nápovedy, `program_usage`, ktorá stručne popisuje použitie, resp. spustenie programu a jej volanie sa prevádza v prípade, že zlyhala kontrola argumentov. Funkcia okrem nápovedy na chybový výstup vypíše aj zoznam dostupných rozhraní, ktoré možno dosadiť za prepínač `-i`.

4.5 printer.c

Hlavnou funkciou súboru `printer.c` je `print_connections_stats`, ktorá sa využíva pre výpis štatistík. K formátovaniu výpisu štatistík je využitá knižnica `ncurses.h` a dáta štatistík sú získavané z hashovacej tabuľky.

Funkcia `get_top_connections` je využitá pre zoradenie spojení zostupne na základe počtu prenesených bitov alebo paketov (udané prepínačom `-s`). Všetky spojenia z tabuľky sú najskôr uložené do pomocného poľa a následne zoradené využitím `Bubble sort` algoritmu. Následne ukazatele (maximálne) desiatich najrušnejších spojení uložené do výstupného (globálneho) poľa `top_connections`.

5 Testovanie

Zdrojové kódy neobsahujú žiadne testovacie súbory, s ktorými by sa dali konkrétne testy znovuopakovať. Proces testovania bol založený prevažne na pozorovaní výstupov aplikácií s podobnou funkcionalitou ako nástroj `isa-top` alebo iných, ktoré dovoľujú prístup k informáciám o paketoch prenášaných po sieti. Takýmito nástrojmi boli napríklad `iftop` a `Wireshark`.

5.1 Porovnanie výstupov s nástrojom iftop

Nástroj `iftop`, podobne ako `isa-top`, umožňuje monitorovať sieťovú aktivitu v reálnom čase a sledovať prenosové rýchlosti aktuálnych spojení. Testovanie spočívalo v spustení viac, či menej rušného prevozu na sieti a porovnaní podobnosti výstupov oboch nástrojov. Najmä pri rušnejšom prevoze však nemuseli byť výstupy totožné. Dôvodom nezhôd medzi výstupmi oboch nástrojov je nemožnosť synchronizovať spustenie oboch nástrojov, čo môže ovplyvniť počet zachytených bajtov a paketov.

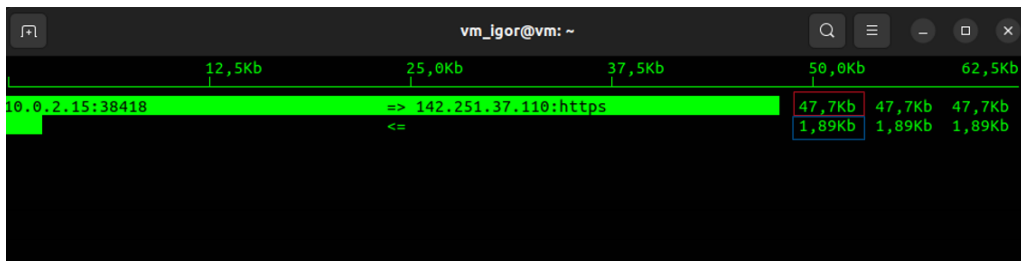
1. Spustenie každého z nástrojov v samostatnom terminále

```
sudo iftop -i enp0s3 -n.  
sudo ./isa-top -i enp0s3 -t 2
```

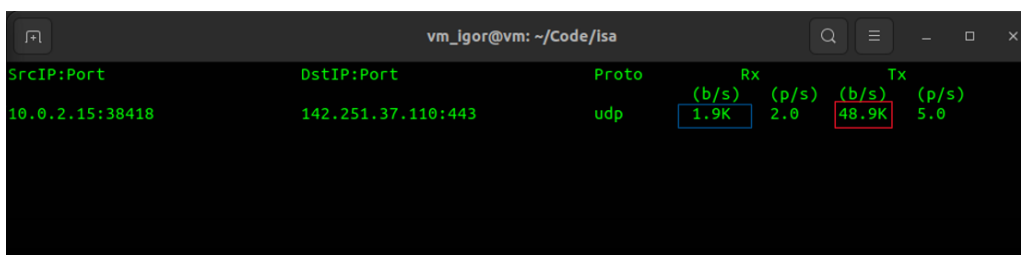
Dôvodom pre prepínač `-t 2` je, že nástroj `iftop` aktualizuje svoje štatistiky každé 2 sekundy a preto potrebujeme zaručiť aby sa aj štatistiky nástroja `isa-top` aktualizovali každé 2 sekundy. Prepínač `-n 2` pri spustení `iftop` zaručí aby výsledné adresy zapisovali číslami IP adresy a nie doménovým menom. Aby sme boli v `iftop` schopní zobrazíť aj čísla portov, musíme ešte po jeho spustení stlačiť klávesu `'P'`.

2. Porovnanie výsledkov

Po spustení oboch nástrojov bol sledovaný výpis štatistík v reálnom čase. Ako ukážka slúži výsledok testovania zobrazený na obrázkoch 1 a 2.



Obr. 1: Výstup nástroja `iftop` počas testovania



Obr. 2: Výstup nástroja `isa-top` počas testovania

5.2 Ping

Nástroj `ping` bol použitý na vytvorenie aktivity na sieti, ktorá bola následne analyzovaná nástrojom `isa-top`. Pomocou `ping` sme schopní vytvoriť predvídateľné správanie (ako známu veľkosť a frekvenciu paketov) a kontrolovať či sú výsledky súhlasné s očakávaniami.

1. Spustenie ping

Testovanie prebiehalo spustením nástroja `ping` voči lokálnej adrese v samostatnom termináli. Napríklad:

```
ping localhost
ping6 ::1
```

2. Monitorovanie výsledkov v isa-top

Po spustení `ping` bol nástroj `isa-top` spustený v samostatnom termináli:

```
sudo ./isa-top -i lo
```

3. Porovnanie výsledkov

Na základe:

- Počtu paketov: Porovnané s frekvenciou odosielania paketov v nástroji `ping`.
- Počtu zachytených bajtov: Očakávané hodnoty boli vypočítané na základe veľkosti ICMP paketu.